

SEATSYNC

# End-to-End Interview Guide

A code-grounded ownership manual for Agrim Jain

| What this gives you                                                                                                                                                     | Ground truth                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Request traces from entry point to response, concurrency reasoning, data models, failure recovery, infrastructure, CI, benchmark interpretation, and interview answers. | Generated from the current repository on 24 Aug 2026 at commit fd370bc. Java 21 is the required runtime. |

**Read actively:** keep the repository open, follow each file reference, and say every interview answer aloud. The final section contains a seven-day ownership plan.

Portfolio-ready does not mean internet-production-ready. This guide marks that boundary explicitly so you can defend the design without overclaiming.

# How to use this guide

Do not try to memorize the repository. Build a mental model, trace one request at a time, then learn the failure cases. A strong interview answer starts with the invariant, names the authoritative component, walks the normal path, and finishes with what happens when a dependency fails.

## Recommended reading passes

| Pass                   | Goal                                | Sections |
|------------------------|-------------------------------------|----------|
| 1. Mental model        | Explain the project in two minutes  | 1-4      |
| 2. Core correctness    | Defend no double booking            | 5-10     |
| 3. Checkout saga       | Trace payment and compensation      | 11-16    |
| 4. Operations          | Run, debug, and verify it           | 17-22    |
| 5. Interview rehearsal | Answer trade-off questions honestly | 23-26    |

### Ownership rule

For every component, be able to answer: What state does it own? What invariant does it enforce? What calls it? What can fail? How does the system recover?

## Table of contents

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>How to use this guide</b>                         | <b>2</b>  |
| Recommended reading passes . . . . .                 | 2         |
| Table of contents . . . . .                          | 3         |
| <b>1. The project in one sentence</b>                | <b>8</b>  |
| The interview-ready two-minute pitch . . . . .       | 8         |
| What each technology is doing . . . . .              | 8         |
| <b>2. Architecture and ownership</b>                 | <b>9</b>  |
| The most important boundary . . . . .                | 9         |
| Why five services, not more . . . . .                | 9         |
| <b>3. Repository and build structure</b>             | <b>10</b> |
| What Maven does . . . . .                            | 10        |
| <b>4. What happens when the system starts</b>        | <b>11</b> |
| Spring Boot entry points . . . . .                   | 11        |
| Auto-configuration chain . . . . .                   | 11        |
| Container startup . . . . .                          | 11        |
| <b>5. API catalog and request entry points</b>       | <b>12</b> |
| Gateway routing . . . . .                            | 12        |
| HTTP status nuance . . . . .                         | 12        |
| <b>6. Gateway: the first code boundary</b>           | <b>13</b> |
| Entry to exit . . . . .                              | 13        |
| Line-by-line guide: GatewayConfiguration . . . . .   | 13        |
| Rate limits . . . . .                                | 13        |
| <b>7. Read path: viewing an event and seat map</b>   | <b>14</b> |
| Why this read can be stale . . . . .                 | 14        |
| Event-service classes . . . . .                      | 14        |
| <b>8. Core request trace: create a seat hold</b>     | <b>15</b> |
| <b>9. Create-hold code, line by line</b>             | <b>16</b> |
| ReservationController lines 20-26 . . . . .          | 16        |
| ReservationService lines 48-96 . . . . .             | 16        |
| <b>10. Why two concurrent buyers cannot both win</b> | <b>17</b> |

|                                                     |           |
|-----------------------------------------------------|-----------|
| Layer 1: Redis absorbs contention . . . . .         | 17        |
| Layer 2: PostgreSQL decides the winner . . . . .    | 17        |
| Layer 3: partial unique index . . . . .             | 17        |
| <b>11. Idempotency: safe client retries</b>         | <b>18</b> |
| Same key, same payload . . . . .                    | 18        |
| Same key, different payload . . . . .               | 18        |
| Important limitation . . . . .                      | 18        |
| <b>12. Transactional outbox: database to Kafka</b>  | <b>19</b> |
| Why delivery is at least once . . . . .             | 19        |
| Why FOR UPDATE SKIP LOCKED . . . . .                | 19        |
| <b>13. Kafka consumer and read-model projection</b> | <b>20</b> |
| ReservationEventProjector lines 22-43 . . . . .     | 20        |
| Retry and dead-letter behavior . . . . .            | 20        |
| Ordering limitation you should own . . . . .        | 20        |
| <b>14. Hold confirmation, release, and expiry</b>   | <b>21</b> |
| Confirm . . . . .                                   | 21        |
| Release . . . . .                                   | 21        |
| Expiry worker . . . . .                             | 21        |
| <b>15. Core request trace: successful booking</b>   | <b>22</b> |
| <b>16. Booking orchestration, line by line</b>      | <b>23</b> |
| BookingController lines 22-30 . . . . .             | 23        |
| BookingStore.start lines 20-50 . . . . .            | 23        |
| BookingOrchestrator.checkout lines 39-67 . . . . .  | 23        |
| completeAuthorized lines 127-140 . . . . .          | 23        |
| <b>17. Payment simulator and its scenarios</b>      | <b>24</b> |
| PaymentService.authorize . . . . .                  | 24        |
| Refund . . . . .                                    | 24        |
| Why timeout does not mean failure . . . . .         | 24        |
| <b>18. Checkout failure and recovery paths</b>      | <b>25</b> |
| A. Payment declines . . . . .                       | 25        |
| B. Payment outcome is unknown . . . . .             | 25        |
| C. Booking process crashes while PENDING . . . . .  | 25        |
| D. Hold cannot be confirmed after payment . . . . . | 25        |
| E. Refund endpoint is unavailable . . . . .         | 25        |

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>19. Reconciliation worker: why it is safe to retry</b>                    | <b>26</b> |
| Retry safety argument . . . . .                                              | 26        |
| <b>20. gRPC and protobuf end to end</b>                                      | <b>27</b> |
| Build time . . . . .                                                         | 27        |
| Server side . . . . .                                                        | 27        |
| Client side . . . . .                                                        | 27        |
| Why gRPC here . . . . .                                                      | 27        |
| <b>21. Data model and database constraints</b>                               | <b>28</b> |
| Four logical databases on one PostgreSQL container . . . . .                 | 28        |
| Why amountMinor is a long . . . . .                                          | 28        |
| Why status is a string enum . . . . .                                        | 28        |
| Locking tools used . . . . .                                                 | 28        |
| <b>22. Docker Compose and container images</b>                               | <b>29</b> |
| Compose topology . . . . .                                                   | 29        |
| Kafka listeners . . . . .                                                    | 29        |
| Dockerfile stages . . . . .                                                  | 29        |
| <b>23. How to run and verify locally</b>                                     | <b>30</b> |
| Option A: infrastructure in containers, Java services in terminals . . . . . | 30        |
| Option B: everything in Compose . . . . .                                    | 30        |
| Health checks . . . . .                                                      | 30        |
| Safe static verification without containers . . . . .                        | 30        |
| Clean shutdown . . . . .                                                     | 30        |
| <b>24. Manual end-to-end walkthrough</b>                                     | <b>31</b> |
| 1. Read seeded data . . . . .                                                | 31        |
| 2. Hold seat A-1 . . . . .                                                   | 31        |
| 3. Book the held seat . . . . .                                              | 31        |
| 4. Observe asynchronous projection . . . . .                                 | 31        |
| 5. Repeat requests . . . . .                                                 | 31        |
| <b>25. GitHub Actions: what runs in the cloud</b>                            | <b>32</b> |
| Build workflow on every push to main and every pull request . . . . .        | 32        |
| Where the technology comes from . . . . .                                    | 32        |
| Why six checks appear . . . . .                                              | 32        |
| Dependabot branches . . . . .                                                | 32        |
| Current evidence . . . . .                                                   | 32        |

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>26. Contention benchmark: what it proves</b>                | <b>33</b> |
| How to launch . . . . .                                        | 33        |
| Hosted execution path . . . . .                                | 33        |
| k6 script details . . . . .                                    | 33        |
| What success does and does not mean . . . . .                  | 33        |
| <b>27. Observability and operations</b>                        | <b>34</b> |
| Implemented hooks . . . . .                                    | 34        |
| Signals to monitor . . . . .                                   | 34        |
| Scaling statement . . . . .                                    | 34        |
| <b>28. Failure matrix you should be able to explain</b>        | <b>35</b> |
| Consistency vs availability choice . . . . .                   | 35        |
| <b>29. Security boundaries</b>                                 | <b>36</b> |
| Threat-oriented reasoning . . . . .                            | 36        |
| <b>30. Known limitations and defensible improvements</b>       | <b>37</b> |
| How to answer 'Why did you not add X?' . . . . .               | 37        |
| <b>31. Debugging playbook</b>                                  | <b>38</b> |
| Useful commands . . . . .                                      | 38        |
| Database questions to ask . . . . .                            | 38        |
| <b>32. Interview questions and strong answers</b>              | <b>39</b> |
| Q1. How do you prevent double booking? . . . . .               | 39        |
| Q2. Why not use only Redis? . . . . .                          | 39        |
| Q3. Why both Redis and PostgreSQL? . . . . .                   | 39        |
| Q4. What is the transactional outbox solving? . . . . .        | 39        |
| Q5. Is Kafka exactly once? . . . . .                           | 39        |
| Q6. Why gRPC? . . . . .                                        | 39        |
| <b>33. Interview questions: failures and scaling</b>           | <b>40</b> |
| Q7. What happens when payment times out? . . . . .             | 40        |
| Q8. What happens if booking crashes mid-request? . . . . .     | 40        |
| Q9. What if payment succeeds after the hold expires? . . . . . | 40        |
| Q10. What happens if Kafka is down? . . . . .                  | 40        |
| Q11. How does it scale? . . . . .                              | 40        |
| Q12. What would you improve first? . . . . .                   | 40        |
| <b>34. Interview questions: design judgment</b>                | <b>41</b> |

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| Q13. Why microservices for a personal project? . . . . .         | 41        |
| Q14. Is the event service the source of truth? . . . . .         | 41        |
| Q15. Why fail closed when Redis is down? . . . . .               | 41        |
| Q16. What does the benchmark demonstrate? . . . . .              | 41        |
| Q17. What is eventual consistency here? . . . . .                | 41        |
| Q18. Where are your transaction boundaries? . . . . .            | 41        |
| <b>35. Whiteboard narratives</b>                                 | <b>42</b> |
| Five-minute no-double-booking narrative . . . . .                | 42        |
| Five-minute checkout narrative . . . . .                         | 42        |
| Thirty-second architecture summary . . . . .                     | 42        |
| <b>36. File-by-file reading map: platform, gateway, event</b>    | <b>43</b> |
| <b>37. File-by-file reading map: reservation</b>                 | <b>44</b> |
| <b>38. File-by-file reading map: booking and payment</b>         | <b>45</b> |
| <b>39. File-by-file reading map: infrastructure and evidence</b> | <b>46</b> |
| <b>40. Glossary</b>                                              | <b>47</b> |
| <b>41. Seven-day ownership plan</b>                              | <b>48</b> |
| Final ownership checklist . . . . .                              | 48        |
| Repository reference . . . . .                                   | 48        |

# 1. The project in one sentence

SeatSync is a five-service Java 21 ticket reservation platform whose central correctness goal is: **one seat can have at most one successful active hold and at most one confirmed booking**, even under concurrent requests, retries, timeouts, and hold expiry.

## The interview-ready two-minute pitch

I built SeatSync to explore correctness under hot-seat contention. A gateway routes clients and applies correlation, admission control, and optional JWT validation. The reservation service is the seat authority. Redis is only a fast atomic gate; PostgreSQL performs the durable conditional state transition and enforces constraints. Reservation changes and outbox records commit together, then a publisher sends at-least-once Kafka events to an idempotent event-service projector. Checkout is coordinated by the booking service: it creates an idempotent booking, calls a deterministic payment simulator, then confirms or releases the hold over gRPC. Unknown outcomes and interrupted PENDING bookings are reconciled by a scheduled worker, while failed confirmation triggers an idempotent refund. Docker Compose runs the stack, Prometheus scrapes actuator metrics, and GitHub Actions compile Java 21, build all service images, and run an opt-in same-seat contention benchmark.

## What each technology is doing

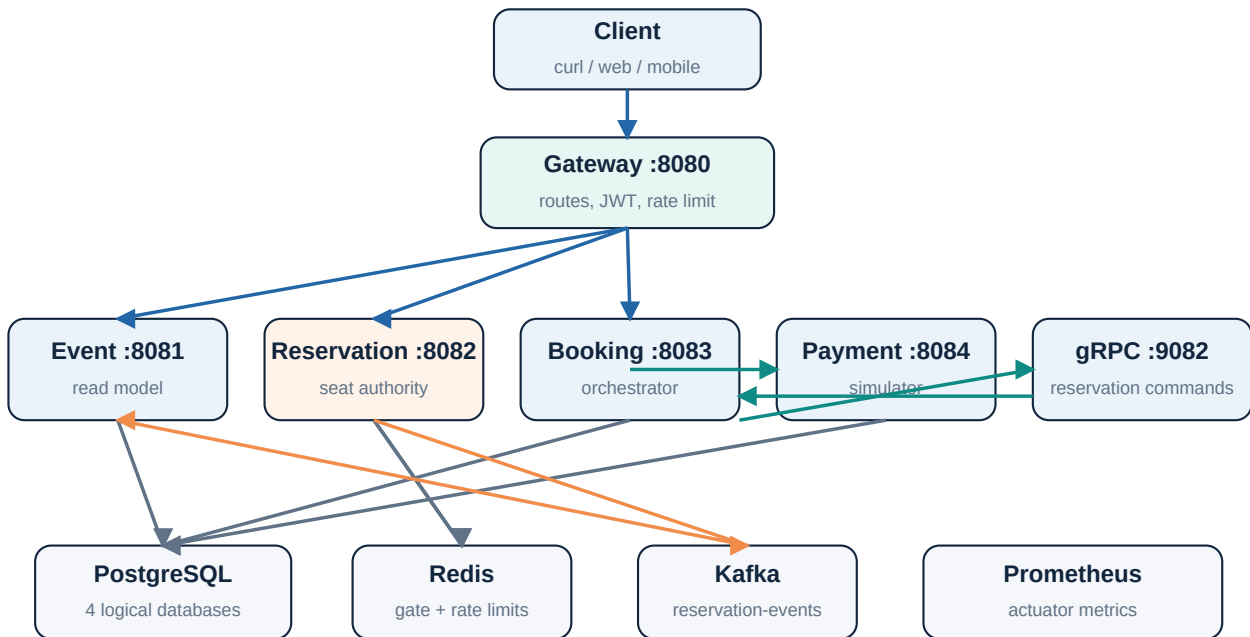
| Technology            | Job in SeatSync                          | Why it exists                                               |
|-----------------------|------------------------------------------|-------------------------------------------------------------|
| Java 21 + Spring Boot | Five independently runnable applications | Clear service boundaries, DI, HTTP, scheduling, data access |
| PostgreSQL            | Authoritative durable state              | Transactions, conditional updates, uniqueness, row locks    |
| Redis                 | Fast seat gate and gateway rate limiter  | Atomic Lua operations and short-lived keys                  |
| Kafka                 | Reservation state-change delivery        | Decouples writes from the seat-map projection               |
| gRPC + protobuf       | Booking to reservation commands          | Typed internal synchronous contract with deadlines          |
| REST                  | Public APIs and booking to payment       | Simple HTTP interaction and OpenAPI support                 |
| Flyway                | Creates and validates schemas            | Versioned database migrations                               |
| Prometheus            | Scrapes actuator metrics                 | Operational visibility hooks                                |
| Docker Compose        | Local and CI topology                    | Repeatable service/dependency startup                       |
| k6                    | Same-seat contention workload            | Validates exactly one winner plus response expectations     |

### Do not overclaim

The repository implements a production-style learning system, not a real payment platform. It has no real card integration, no full automated test suite, no public-production secret setup, and no claim of internet-scale capacity.



## 2. Architecture and ownership



*Solid request paths above; infrastructure and asynchronous paths below*

| Service             | Port                  | Owns                                                   | Does not own                 |
|---------------------|-----------------------|--------------------------------------------------------|------------------------------|
| gateway-service     | 8080                  | Routing policy, correlation, rate-limit decisions      | Business state               |
| event-service       | 8081                  | Event catalog and eventually consistent seat-map view  | Reservation truth            |
| reservation-service | 8082 REST / 9082 gRPC | Seat inventory, holds, idempotency, reservation outbox | Payments                     |
| booking-service     | 8083                  | Booking workflow state                                 | Seat truth or payment ledger |
| payment-service     | 8084                  | Simulated payment state                                | Real money movement          |

### The most important boundary

The event service may display stale availability because Kafka delivery is asynchronous. That cannot create overselling because every hold still goes to the reservation service, which checks its own PostgreSQL inventory. The write model is authoritative; the read model is optimized for queries.

### Why five services, not more

The split follows business ownership: catalog/read model, reservation consistency, booking orchestration, payments, and edge policy. The shared platform-contracts module is a library, not a deployable service. This is enough to demonstrate distributed workflows while keeping the code explainable.

#### Interview phrase

I split services by data ownership and failure boundary, not by technical layer. Each deployable service owns its database schema and can fail independently.

### 3. Repository and build structure

The root `pom.xml` is a Maven reactor. Packaging is `pom`, so it produces no application itself; it builds the shared contract module and five service modules in dependency order.

```
<modules>
  <module>platform-contracts</module>
  <module>gateway-service</module>
  <module>event-service</module>
  <module>reservation-service</module>
  <module>booking-service</module>
  <module>payment-service</module>
</modules>

<java.version>21</java.version>
<spring-boot.version>3.4.5</spring-boot.version>
```

| Path                                            | Meaning                                                                |
|-------------------------------------------------|------------------------------------------------------------------------|
| <code>pom.xml</code>                            | Central versions, module order, Java 21 compiler release               |
| <code>&lt;service&gt;/pom.xml</code>            | Only the dependencies needed by that service                           |
| <code>src/main/java</code>                      | Spring application and business code                                   |
| <code>src/main/resources/application.yml</code> | Ports, dependency addresses, timeouts, actuator exposure               |
| <code>src/main/resources/db/migration</code>    | Flyway-owned schema and seed data                                      |
| <code>platform-contracts/src/main/proto</code>  | gRPC schema compiled into Java stubs                                   |
| <code>compose.yml</code>                        | Nine-container local topology including dependencies and observability |
| <code>.github/workflows</code>                  | Hosted build/container verification and manual benchmark               |
| <code>load-tests</code>                         | k6 same-seat contention scenario                                       |
| <code>docs</code>                               | Invariants, ADRs, operations, and manual verification                  |

#### What Maven does

- Dependency management fixes compatible Spring Boot, Spring Cloud, gRPC, and protobuf versions.
- The compiler plugin uses `release 21`. A Java 17 JDK cannot compile it.
- The protobuf plugin downloads platform-specific `protoc` and the gRPC Java generator.
- Each service uses Spring Boot's `repackage` goal to create an executable fat JAR.
- `mvn -B -DskipTests package` compiles and packages; tests are explicitly skipped and currently no test sources exist.

## 4. What happens when the system starts

### Spring Boot entry points

| Application class             | Startup behavior                                                    |
|-------------------------------|---------------------------------------------------------------------|
| GatewayApplication            | Starts reactive Spring Cloud Gateway on 8080                        |
| EventServiceApplication       | Starts MVC/JPA/Kafka listener application on 8081                   |
| ReservationServiceApplication | Starts MVC/JPA/Redis/Kafka and enables scheduled expiry/outbox work |
| BookingServiceApplication     | Starts MVC/JPA/gRPC client and enables reconciliation scheduling    |
| PaymentServiceApplication     | Starts MVC/JPA payment simulator on 8084                            |

### Auto-configuration chain

#### For each stateful service

| # | From        | To                | Action / code boundary                                              |
|---|-------------|-------------------|---------------------------------------------------------------------|
| 1 | java -jar   | SpringApplication | Create application context and scan the service package             |
| 2 | Spring Boot | DataSource        | Read DB_URL/DB_USER/DB_PASSWORD and create a connection pool        |
| 3 | Flyway      | PostgreSQL        | Apply V1 migration if needed                                        |
| 4 | Hibernate   | Schema            | Validate entity mappings because ddl-auto=validate                  |
| 5 | Spring      | Beans             | Construct controllers, services, repositories, clients, and workers |
| 6 | Actuator    | HTTP              | Expose health, info, and Prometheus endpoints                       |

### Container startup

Compose first starts PostgreSQL, Redis, and Kafka. PostgreSQL and Redis have health checks. Service containers receive internal DNS names such as postgres, redis, and kafka. The same configuration defaults to localhost when services run directly on the laptop.

#### Why Flyway plus validate

Flyway owns schema evolution. Hibernate only verifies that Java entities match the schema. This avoids accidental production schema changes from ORM auto-generation.

## 5. API catalog and request entry points

| Method and path                                    | Entry class                         | Outcome                      |
|----------------------------------------------------|-------------------------------------|------------------------------|
| GET /api/events                                    | EventController.list                | All seeded events            |
| GET /api/events/{eventId}                          | EventController.get                 | Event or 404                 |
| GET /api/events/{eventId}/seats                    | EventController.seatMap             | Ordered seat-map projection  |
| POST /api/reservations/holds                       | ReservationController.create        | 201 hold or 4xx ApiError     |
| GET /api/reservations/holds/{id}                   | ReservationController.get           | Current authoritative hold   |
| POST /api/reservations/holds/{id}/confirm          | ReservationController.confirm       | Confirmed hold               |
| DELETE /api/reservations/holds/{id}?customerId=... | ReservationController.release       | 204 after idempotent release |
| POST /api/bookings                                 | BookingController.create            | Booking workflow result      |
| GET /api/bookings/{id}                             | BookingController.get               | Current booking state        |
| POST /internal/payments/authorize                  | PaymentController.authorize         | Internal payment result      |
| GET /internal/payments?bookingId=...               | PaymentController.byBooking         | Reconciliation lookup        |
| POST /internal/payments/{id}/refund                | PaymentController.refund            | Idempotent refund            |
| gRPC ConfirmHold                                   | ReservationGrpcEndpoint.confirmHold | Typed internal confirmation  |
| gRPC ReleaseHold                                   | ReservationGrpcEndpoint.releaseHold | Typed internal release       |

### Gateway routing

The client normally calls port 8080. Path predicates route events to 8081, reservations to 8082, and bookings to 8083 without rewriting the path. Payment endpoints are intentionally not routed through the gateway; booking calls them on the internal network.

### HTTP status nuance

A new booking request returns HTTP 201 because the controller checks that the newly persisted object was PENDING before orchestration. The body may already be CONFIRMED, FAILED, or PAYMENT\_UNKNOWN. An idempotent replay returns HTTP 200. This is explainable but unconventional; a production API might instead return 200 for a completed synchronous result or 202 for an ongoing workflow.

## 6. Gateway: the first code boundary

### Entry to exit

#### Example: POST /api/reservations/holds

| # | From                 | To                 | Action / code boundary                                                        |
|---|----------------------|--------------------|-------------------------------------------------------------------------------|
| 1 | Client               | Gateway :8080      | Send JSON plus Idempotency-Key; optionally Authorization and X-Correlation-ID |
| 2 | Dev/ProdSecurity     | Request            | Permit all in dev or validate JWT for /api/** in prod                         |
| 3 | correlationFilter    | Request/response   | Reuse correlation ID or create UUID; add it to both directions                |
| 4 | customerKeyResolver  | Redis rate limiter | Resolve principal, then auth header, then IP, then anonymous                  |
| 5 | Gateway route        | Reservation :8082  | Proxy the unchanged path and headers                                          |
| 6 | Reservation response | Client             | Gateway streams status/body and returns X-Correlation-ID                      |

### Line-by-line guide: GatewayConfiguration

| Lines                   | Meaning                                                                                                    |
|-------------------------|------------------------------------------------------------------------------------------------------------|
| 13                      | Defines X-Correlation-ID once to avoid duplicated string literals                                          |
| 15-25                   | GlobalFilter reads the inbound header, generates UUID if absent, mutates request, and sets response header |
| 28-40                   | KeyResolver chooses the rate-limit identity in decreasing order of trust/context                           |
| application.yml 7-29    | Path routes and Redis token-bucket capacities                                                              |
| DevSecurity 7-15        | Non-prod profile disables CSRF and permits all routes                                                      |
| ProductionSecurity 7-22 | Prod permits health, authenticates /api/** with JWT, denies everything else                                |

### Rate limits

| Route        | Replenish       | Burst | Interpretation                                                |
|--------------|-----------------|-------|---------------------------------------------------------------|
| Reservations | 20 tokens/sec   | 40    | Short bursts up to 40, then refill at 20/sec per resolved key |
| Bookings     | 10 tokens/sec   | 20    | Lower checkout admission rate                                 |
| Events       | None configured | None  | Read route is not rate limited in current code                |

#### Security truth

JWT is implemented only when the prod profile is active and JWT\_ISSUER\_URI is set. Internal service ports are exposed in Compose for debugging and have no service-to-service authentication.

## 7. Read path: viewing an event and seat map

### GET /api/events/{eventId}/seats

| # | From                     | To              | Action / code boundary                            |
|---|--------------------------|-----------------|---------------------------------------------------|
| 1 | Client                   | Gateway         | GET /api/events/{eventId}/seats                   |
| 2 | Gateway                  | EventController | Route by Path=/api/events/**                      |
| 3 | EventController.seat Map | SeatRepository  | Derived query ordered by section, rowLabel, label |
| 4 | JPA                      | seats table     | Read the event-service projection database        |
| 5 | Spring MVC/Jackson       | Client          | Serialize List<Seat> as JSON                      |

### Why this read can be stale

The event service updates availability only after it consumes a Kafka reservation event. Immediately after a hold commits, the reservation database says HELD while the event database may still say AVAILABLE. This is eventual consistency. A client seeing AVAILABLE may attempt a hold, but the reservation service still rejects it from its authoritative database, so stale reads affect user experience, not the no-oversell invariant.

### Event-service classes

| Class                            | Responsibility                                            |
|----------------------------------|-----------------------------------------------------------|
| Event                            | JPA entity for name, venue, sale and start times          |
| Seat                             | JPA read-model entity with AVAILABLE/HELD/BOOKED          |
| EventController                  | Three read-only HTTP endpoints                            |
| EventRepository / SeatRepository | Spring Data access plus projection update query           |
| ReservationEventProjector        | Kafka consumer, inbox deduplication, availability mapping |
| KafkaFailureConfiguration        | Three retries with 250 ms backoff, then topic.DLT         |

## 8. Core request trace: create a seat hold

```
POST /api/reservations/holds
Idempotency-Key: demo-hold-1
{
  "eventId": "10000000-...-0001",
  "seatId": "20000000-...-0001",
  "customerId": "30000000-...-0001"
}
```

### Entry point to HTTP response

| #  | From        | To                            | Action / code boundary                                                |
|----|-------------|-------------------------------|-----------------------------------------------------------------------|
| 1  | Gateway     | ReservationController.create  | Spring binds header/body; Jakarta validation requires all UUIDs       |
| 2  | Controller  | ReservationService.createHold | Pass key and typed CreateHoldRequest                                  |
| 3  | Service     | idempotency_records           | Require key, hash payload, return prior hold for exact replay         |
| 4  | Service     | RedisSeatGate.acquire         | Atomic SET-if-absent with ownership token and TTL                     |
| 5  | Service     | seat_inventory                | Conditional AVAILABLE -> HELD update; affected-row count must equal 1 |
| 6  | Service     | seat_holds                    | Insert ACTIVE hold with owner and expiry                              |
| 7  | Service     | idempotency_records           | Insert key -> request hash -> hold ID                                 |
| 8  | Service     | outbox_events                 | Insert reservation.hold.created payload                               |
| 9  | Transaction | PostgreSQL                    | Commit inventory, hold, idempotency, and outbox atomically            |
| 10 | Controller  | Client                        | Return 201 Created, Location header, and HoldResponse JSON            |

#### Transaction boundary

ReservationService.createHold is @Transactional. PostgreSQL changes either all commit or all roll back. Redis is external to that transaction, so an after-completion callback removes the caller's token if the database transaction rolls back.

## 9. Create-hold code, line by line

### ReservationController lines 20-26

```
@PostMapping
ResponseEntity<HoldResponse> create(
    @RequestHeader(name = "Idempotency-Key", required = false) String key,
    @Valid @RequestBody CreateHoldRequest request) {
    HoldResponse response = reservations.createHold(key, request);
    return ResponseEntity.created(
        URI.create("/api/reservations/holds/" + response.holdId()))
        .body(response);
}
```

- @PostMapping maps POST at the class-level path.
- The header is marked optional at binding time so business code can return a domain-specific error.
- @Valid executes constraints on the record before entering business logic.
- The service returns a DTO, keeping the JPA entity package-private.
- HTTP 201 and Location communicate creation semantics.

### ReservationService lines 48-96

| Lines | Decision                               | Why                                                           |
|-------|----------------------------------------|---------------------------------------------------------------|
| 50-53 | Reject blank idempotency key           | Safe retries need a stable command identity                   |
| 55    | SHA-256 hash eventId:seatId:customerId | Detect same key with a different payload                      |
| 56-67 | Replay lookup                          | Return prior result or reject key misuse                      |
| 69-71 | Create hold ID and Redis token         | The UUID identifies both durable hold and temporary ownership |
| 74-76 | Acquire Redis gate                     | Reject most losers before they hit the hot database row       |
| 77    | Register rollback cleanup              | Do not leave a ghost gate if SQL work fails                   |
| 80-83 | Conditional SQL update                 | This is the durable single-winner boundary                    |
| 85-92 | Create hold, idempotency, outbox       | All three are part of the same transaction                    |
| 93-95 | Translate constraint collision         | Concurrent database winner becomes a 409 domain response      |
| 96    | Create response from entity            | Return committed business data after method exits             |



## 10. Why two concurrent buyers cannot both win

### Layer 1: Redis absorbs contention

The ACQUIRE Lua script checks key existence and writes the token plus TTL in one atomic Redis execution. Only one request receives 1. This is fast, but it is not trusted as durable truth.

```
if redis.call('exists', KEYS[1]) == 0 then
    redis.call('psetex', KEYS[1], ARGV[2], ARGV[1])
    return 1
else
    return 0
end
```

### Layer 2: PostgreSQL decides the winner

```
UPDATE seat_inventory
SET state='HELD', version=version+1
WHERE seat_id=:seatId
AND event_id=:eventId
AND state='AVAILABLE'
```

PostgreSQL locks and rechecks the row during concurrent updates. Exactly one request can change AVAILABLE to HELD; later contenders affect zero rows and receive SEAT\_UNAVAILABLE.

### Layer 3: partial unique index

```
CREATE UNIQUE INDEX uk_active_hold_per_seat
ON seat_holds(seat_id) WHERE status = 'ACTIVE';
```

Even if application logic regresses, the database rejects two ACTIVE hold rows for one seat.

| Failure                   | What happens                                                            | Invariant |
|---------------------------|-------------------------------------------------------------------------|-----------|
| Redis key disappears      | A contender passes Redis but PostgreSQL sees HELD/BOOKED and rejects it | Preserved |
| Redis unavailable         | Acquire throws; request fails closed before SQL mutation                | Preserved |
| DB transaction rolls back | After-completion callback compare-deletes only this token               | Preserved |
| Wrong token tries cleanup | RELEASE Lua script compares value before delete                         | Preserved |
| Many service instances    | Redis script is atomic and DB constraint is shared                      | Preserved |

#### Best interview answer

Redis improves admission latency; PostgreSQL guarantees correctness. If Redis is wrong, the system may reject extra requests or send more load to PostgreSQL, but it still cannot sell the same seat twice.

## 11. Idempotency: safe client retries

A client may retry because it did not receive the response. Without idempotency, the retry could create another hold or charge again. SeatSync stores a stable key with a hash of the command payload.

| Command           | Key storage                        | Payload check                                      | Replay                                                   |
|-------------------|------------------------------------|----------------------------------------------------|----------------------------------------------------------|
| Create hold       | idempotency_records primary key    | eventId + seatId + customerId                      | Return the stored hold                                   |
| Create booking    | bookings.idempotency_key<br>UNIQUE | hold, customer, amount, currency,<br>payment token | Return existing booking and<br>skip checkout if terminal |
| Authorize payment | payments.idempotency_key<br>UNIQUE | booking, amount, currency                          | Return existing payment                                  |
| Refund            | Payment.refund state check         | Payment ID                                         | REFUNDED stays REFUNDED                                  |
| Kafka projection  | event_inbox.event_id primary key   | Event envelope ID                                  | Duplicate exits before projection<br>update              |

### Same key, same payload

Return the original outcome. This is what makes a lost response safe to retry.

### Same key, different payload

Return IDEMPOTENCY\_KEY\_REUSED. Silent reuse would make the caller believe a different command succeeded.

### Important limitation

Two requests with the same new idempotency key can race before the first commit. Database uniqueness prevents duplicate durable records, but the loser may receive a conflict instead of the first result. A production-grade idempotency layer might serialize key creation or store a PROCESSING result and let duplicates wait/poll.

## 12. Transactional outbox: database to Kafka

Writing the reservation row and calling Kafka directly creates a dual-write problem: the database might commit while Kafka fails, or Kafka might publish while the database rolls back. SeatSync writes an outbox row in the same PostgreSQL transaction as the domain change.

### Outbox lifecycle

| # | From               | To            | Action / code boundary                                         |
|---|--------------------|---------------|----------------------------------------------------------------|
| 1 | ReservationService | PostgreSQL    | Commit hold transition and outbox row together                 |
| 2 | OutboxPublisher    | outbox_events | Every 500 ms lock up to 100 unpublished rows using SKIP LOCKED |
| 3 | Publisher          | ObjectMapper  | Wrap event ID, type, schemaVersion, aggregateId, and data      |
| 4 | Publisher          | Kafka         | Send with aggregateId as key and wait up to 2 seconds          |
| 5 | Publisher          | outbox_events | Set published_at after Kafka acknowledgement                   |
| 6 | Transaction        | PostgreSQL    | Commit the published marker                                    |

### Why delivery is at least once

If the process crashes after Kafka acknowledges but before published\_at commits, the row is sent again after restart. SeatSync accepts duplicates and makes the consumer idempotent. It does not claim end-to-end exactly-once delivery.

### Why FOR UPDATE SKIP LOCKED

Multiple reservation instances can run publishers. Each transaction locks a different batch and skips rows already locked by another publisher, increasing throughput without double-claiming the same batch during normal execution.

#### Honest nuance

The shared EventEnvelope record exists, but OutboxPublisher currently builds a Map and does not include occurredAt or correlationId. Correlation reaches downstream HTTP services but is not yet persisted into reservation events.

## 13. Kafka consumer and read-model projection

### ReservationEventProjector lines 22-43

| Lines | Action                                                                 |
|-------|------------------------------------------------------------------------|
| 22-24 | Subscribe to reservation-events with group seat-map-projector          |
| 25    | Wrap inbox insert and seat update in one database transaction          |
| 27-29 | Parse JSON and require eventId/eventType                               |
| 30-35 | Insert event ID with ON CONFLICT DO NOTHING; return on duplicate       |
| 36-39 | Read eventId, seatId, and reservation status from data                 |
| 40-42 | Update exactly one projection row or throw so the transaction retries  |
| 45-50 | Map ACTIVE to HELD, CONFIRMED to BOOKED, RELEASED/EXPIRED to AVAILABLE |

### Retry and dead-letter behavior

A malformed or unprojectable record is retried three times with 250 ms fixed backoff. After retries, `DeadLetterPublishingRecoverer` writes it to `reservation-events.DLT` on the same partition. Operators should alert on any DLT record.

### Ordering limitation you should own

Kafka keys are currently hold IDs, not seat IDs. A later hold for the same seat has a different key and may land on a different partition, so projection events across hold lifecycles are not strictly ordered by seat. The authoritative reservation database remains correct, but the read model could temporarily apply an older event after a newer one. A production improvement is to key by `seatId` and carry a monotonic seat version so the projector rejects stale versions.

#### Interview response

I chose at-least-once delivery with an inbox because network delivery cannot guarantee exactly once. Business effects are made idempotent at the database boundary.

## 14. Hold confirmation, release, and expiry

### Confirm

#### ReservationService.confirm

| # | From        | To                | Action / code boundary                                       |
|---|-------------|-------------------|--------------------------------------------------------------|
| 1 | Caller      | findByIdForUpdate | Pessimistically lock the hold row                            |
| 2 | Service     | Owner check       | Reject a different customer                                  |
| 3 | Service     | Status check      | Return confirmed state idempotently; reject expired/released |
| 4 | Service     | seat_inventory    | Conditional HELD -> BOOKED                                   |
| 5 | Service     | SeatHold          | Set CONFIRMED                                                |
| 6 | Service     | Redis             | Compare-delete the ownership key                             |
| 7 | Service     | outbox            | Record reservation.hold.confirmed                            |
| 8 | Transaction | PostgreSQL        | Commit inventory, hold, and outbox together                  |

### Release

Release uses the same hold row lock and owner check. If the hold is no longer ACTIVE, it returns without changing state. Otherwise it changes inventory HELD to AVAILABLE, marks the hold RELEASED, removes the Redis token, and writes an outbox event.

### Expiry worker

Every second, HoldExpiryScheduler finds up to 100 ACTIVE holds past expiresAt. Each ID is passed into a separate transactional HoldExpiryProcessor method. The processor locks the row and rechecks status/time, preventing a race with confirmation. It releases inventory, marks EXPIRED, removes Redis, and writes the outbox event.

| Race                                          | Winner behavior                                                                 |
|-----------------------------------------------|---------------------------------------------------------------------------------|
| Confirm obtains hold lock first               | It sees ACTIVE/unexpired, commits BOOKED; expiry later sees CONFIRMED and exits |
| Expiry obtains hold lock first after deadline | It commits AVAILABLE/EXPIRED; confirm later rejects HOLD_NOT_ACTIVE             |
| Release called twice                          | First releases; second sees non-ACTIVE and returns                              |
| Confirm called twice by owner                 | First confirms; second returns confirmed response                               |

## 15. Core request trace: successful booking

```
POST /api/bookings
Idempotency-Key: demo-booking-1
{
  "holdId": "<hold-id>",
  "customerId": "30000000-...-0001",
  "amountMinor": 499900,
  "currency": "INR",
  "paymentMethodToken": "pm_success"
}
```

### Entry point to confirmed booking

| #  | From               | To                           | Action / code boundary                         |
|----|--------------------|------------------------------|------------------------------------------------|
| 1  | Gateway            | BookingController.create     | Validate request and capture Idempotency-Key   |
| 2  | Controller         | BookingStore.start           | Hash request; replay or insert PENDING booking |
| 3  | BookingStore       | PostgreSQL                   | Commit PENDING before any external call        |
| 4  | Controller         | BookingOrchestrator.checkout | Continue only when status is PENDING           |
| 5  | Orchestrator       | Payment REST                 | POST authorize with key booking-{bookingId}    |
| 6  | PaymentService     | payments table               | Simulate 30 ms then persist AUTHORIZED         |
| 7  | Orchestrator       | Reservation gRPC             | ConfirmHold with 2-second deadline             |
| 8  | ReservationService | PostgreSQL                   | Lock hold, HELD -> BOOKED, CONFIRMED, outbox   |
| 9  | Orchestrator       | BookingStore                 | Set booking CONFIRMED and paymentId            |
| 10 | Controller         | Client                       | Return BookingResponse JSON                    |

#### Why persist PENDING first

It creates a durable saga checkpoint before calling another service. If the booking process stops after this commit, the scheduled worker can find and reconcile the unfinished booking.

## 16. Booking orchestration, line by line

### BookingController lines 22-30

- `store.start` runs in its own proxied transaction and returns a durable Booking.
- New PENDING objects produce HTTP 201; replays produce HTTP 200.
- `orchestrator.checkout` performs network calls outside the start transaction.

### BookingStore.start lines 20-50

- Require the client idempotency key.
- Hash hold, customer, amount, currency, and simulator token.
- If key exists and hash differs, reject. If equal, return prior booking.
- Insert a new UUID booking in PENDING. The database also enforces unique key and unique holdId.

### BookingOrchestrator.checkout lines 39-67

| Lines | Behavior                                                                      |
|-------|-------------------------------------------------------------------------------|
| 40    | Terminal replay exits immediately; no repeated payment call                   |
| 42-55 | Call payment authorize with service-generated idempotency key                 |
| 56-59 | Any REST exception becomes PAYMENT_UNKNOWN because the outcome may be unknown |
| 60-64 | Null/non-AUTHORIZED result releases hold and marks PAYMENT_DECLINED           |
| 65    | AUTHORIZED result enters completeAuthorized                                   |
| 66    | Reload and return current durable booking state                               |

### completeAuthorized lines 127-140

The booking service calls typed gRPC ConfirmHold with the durable hold/customer IDs and a two-second deadline. Success marks the booking CONFIRMED. Any exception records REFUND\_PENDING and immediately attempts a refund; the worker will retry if that attempt fails.

#### Simplification to disclose

The catch is broad. A production saga should distinguish a definite hold rejection from an unknown gRPC outcome and should query/retry confirmation before refunding, because the server may have confirmed just before the response timed out.

## 17. Payment simulator and its scenarios

| Token prefix | Simulator behavior                                        | Booking result                       |
|--------------|-----------------------------------------------------------|--------------------------------------|
| pm_success   | Sleep about 30 ms and persist AUTHORIZED                  | Confirm hold, then CONFIRMED         |
| pm_decline   | Sleep about 30 ms and persist DECLINED                    | Release hold, then FAILED            |
| pm_timeout   | Sleep 4 seconds; booking client times out after 3 seconds | PAYMENT_UNKNOWN, then reconciliation |

### PaymentService.authorize

- Require Idempotency-Key.
- Return an existing payment if key, booking, amount, and currency agree.
- Sleep only to simulate provider latency. This is not a real provider integration.
- Choose DECLINED only for pm\_decline; every other token authorizes.
- Persist one payment per booking using both idempotency\_key and booking\_id uniqueness.

### Refund

Payment.refund changes AUTHORIZED to REFUNDED. Calling it again leaves REFUNDED unchanged and returns the same state, so compensation is idempotent.

### Why timeout does not mean failure

The booking client's read timeout is three seconds, while pm\_timeout finishes after four. The client stops waiting, but the payment service thread continues and commits AUTHORIZED. Therefore the booking service must not assume decline; it stores PAYMENT\_UNKNOWN and later asks by bookingId.

#### Production difference

A real integration would use provider tokens/webhooks, secure network and secret handling, precise provider error classification, and a ledger/reconciliation process. Thread.sleep inside a database transaction is intentionally simulator-only and would waste connections under load.



## 18. Checkout failure and recovery paths

### A. Payment declines

Payment returns DECLINED. Booking sends gRPC ReleaseHold, marks FAILED with PAYMENT\_DECLINED, and returns. If release cannot reach reservation, the hold remains protected and eventually expires.

### B. Payment outcome is unknown

Any exception from authorize sets PAYMENT\_UNKNOWN. Every two seconds the worker queries GET /internal/payments?bookingId=... . AUTHORIZED leads to confirmation; DECLINED leads to release plus failure; repeated 404 after the 10-second grace leads to release plus PAYMENT\_NOT\_FOUND.

### C. Booking process crashes while PENDING

The worker scans PENDING rows older than PAYMENT\_RESULT\_GRACE. Waiting ten seconds prevents overlap with a normal in-flight checkout. If a payment exists, it continues the workflow. If none exists, it conservatively fails and releases the hold because the original payment token is not stored for retry.

### D. Hold cannot be confirmed after payment

Booking records paymentId, status REFUND\_PENDING, and failure HOLD\_CONFIRMATION\_FAILED. It calls refund immediately. A successful refund changes booking to REFUNDED; failure leaves REFUND\_PENDING for the worker.

### E. Refund endpoint is unavailable

retryRefund logs the exception and keeps durable REFUND\_PENDING. The next worker scan repeats the idempotent refund.

| Booking state   | Meaning                                        | Next actor                       |
|-----------------|------------------------------------------------|----------------------------------|
| PENDING         | Durable checkpoint before known payment result | Checkout or stale-PENDING worker |
| PAYMENT_UNKNOWN | Authorize call outcome unclear                 | Reconciliation lookup            |
| CONFIRMED       | Payment authorized and hold confirmed          | Terminal                         |
| FAILED          | No successful payment should remain            | Terminal                         |
| REFUND_PENDING  | Payment exists but booking cannot complete     | Immediate/worker refund          |
| REFUNDED        | Compensation completed                         | Terminal                         |

## 19. Reconciliation worker: why it is safe to retry

```
@Scheduled(fixedDelayString = "${seatsync.payment-reconciliation-ms:2000}")
void reconcile() {
    store.pendingPaymentsBefore(Instant.now().minus(paymentResultGrace))
        .forEach(orchestrator::reconcile);
    store.unknownPayments().forEach(orchestrator::reconcile);
    store.pendingRefunds().forEach(orchestrator::retryRefund);
}
```

| Property                     | Default   | Purpose                                                |
|------------------------------|-----------|--------------------------------------------------------|
| PAYMENT_RECONCILIATION_MS    | 2000 ms   | Delay between completed scans                          |
| PAYMENT_RESULT_GRACE         | PT10S     | Do not treat a currently running checkout as abandoned |
| Payment REST connect timeout | 500 ms    | Fail quickly when service cannot be reached            |
| Payment REST read timeout    | 3000 ms   | Create an unknown-outcome scenario for pm_timeout      |
| Reservation gRPC deadline    | 2 seconds | Bound synchronous confirmation/release waiting         |

### Retry safety argument

- Payment authorization uses booking-{bookingId}; repeating does not create another payment.
- Payment lookup is read-only.
- Reservation confirmation is idempotent for the same confirmed hold and owner.
- Reservation release is idempotent once the hold is no longer ACTIVE.
- Payment refund is idempotent once status is REFUNDED.
- Booking transitions overwrite the same booking row rather than creating a new record.

#### Multi-instance nuance

Workers do not claim booking rows with SKIP LOCKED, so two booking instances could reconcile the same row. Downstream operations are mostly idempotent, but a stricter production design would add row claiming/locking or optimistic version checks.

## 20. gRPC and protobuf end to end

```
service ReservationCommandService {
  rpc ConfirmHold (ConfirmHoldRequest) returns (HoldReply);
  rpc ReleaseHold (ReleaseHoldRequest) returns (HoldReply);
}

message ConfirmHoldRequest { string hold_id = 1; string customer_id = 2; }
message ReleaseHoldRequest { string hold_id = 1; string customer_id = 2; }
```

### Build time

The protobuf Maven plugin runs `protoc` and `protoc-gen-grpc-java` for the detected OS/CPU. It generates message classes, a client stub, and an abstract server base in `platform-contracts/target/generated-sources`. Booking and reservation depend on that shared JAR.

### Server side

`ReservationGrpcEndpoint` extends the generated `ImplBase`. `GrpcServerConfiguration` starts a Netty gRPC server on 9082 and registers the endpoint. The endpoint parses UUID strings, calls the same `ReservationService` used by REST, converts a domain response to `HoldReply`, and maps `ReservationException` to `FAILED_PRECONDITION`.

### Client side

`GrpcClientConfiguration` creates a plaintext `ManagedChannel` using `RESERVATION_GRPC_HOST/PORT`. Booking creates a blocking stub and attaches a two-second deadline per command.

### Why gRPC here

The command contract is internal, small, synchronous, and typed. Protobuf makes schema fields explicit and generated stubs remove hand-written serialization. REST remains suitable for public APIs and the simple payment simulator.

#### Production boundary

Compose uses plaintext gRPC and no workload identity. A real shared environment should add service authentication and TLS/mTLS.

## 21. Data model and database constraints

### Four logical databases on one PostgreSQL container

| Database              | Tables                                                         | Critical constraints                                             |
|-----------------------|----------------------------------------------------------------|------------------------------------------------------------------|
| seatsync_events       | events, seats, event_inbox                                     | Event-seat label uniqueness; inbox event ID primary key          |
| seatsync_reservations | seat_inventory, seat_holds, idempotency_records, outbox_events | Seat primary key; one ACTIVE hold per seat; idem key primary key |
| seatsync_bookings     | bookings                                                       | Unique client idempotency key and unique holdId                  |
| seatsync_payments     | payments                                                       | Unique payment idempotency key and unique bookingId              |

### Why amountMinor is a long

499900 means INR 4,999.00 if the currency uses two fractional digits. Integer minor units avoid binary floating-point errors. Currency is validated as three uppercase letters, though production money handling would use currency-specific fraction rules.

### Why status is a string enum

JPA EnumType.STRING and SQL CHECK constraints keep database values readable and reject unknown states. Adding a new state requires both Java and Flyway migration changes.

### Locking tools used

| Tool                   | Where                       | Purpose                                                                  |
|------------------------|-----------------------------|--------------------------------------------------------------------------|
| Conditional UPDATE     | seat_inventory              | Single winner without read-then-write race                               |
| Partial UNIQUE index   | seat_holds                  | At most one ACTIVE hold per seat                                         |
| PESSIMISTIC_WRITE      | confirm/release/expire hold | Serialize transitions for one hold                                       |
| FOR UPDATE SKIP LOCKED | outbox batch                | Parallel publishers claim separate rows                                  |
| UNIQUE keys            | booking/payment/idempotency | Exactly one durable command effect                                       |
| @Version               | SeatInventory entity        | Present for ORM optimistic versioning; native updates increment manually |

## 22. Docker Compose and container images

### Compose topology

| Container           | Image/build            | Host ports | Key detail                                                  |
|---------------------|------------------------|------------|-------------------------------------------------------------|
| postgres            | postgres:16-alpine     | 5432       | init.sql creates four databases; named volume persists data |
| redis               | redis:7.4-alpine       | 6379       | AOF enabled; noeviction policy                              |
| kafka               | apache/kafka:3.9.1     | 29092      | Single-node KRaft; internal kafka:9092                      |
| event-service       | Shared Dockerfile      | 8081       | DB + Kafka                                                  |
| reservation-service | Shared Dockerfile      | 8082, 9082 | DB + Redis + Kafka                                          |
| booking-service     | Shared Dockerfile      | 8083       | DB + reservation gRPC + payment REST                        |
| payment-service     | Shared Dockerfile      | 8084       | DB only                                                     |
| gateway-service     | Shared Dockerfile      | 8080       | Redis + downstream HTTP                                     |
| prometheus          | prom/prometheus:v3.2.1 | 9090       | Only with observability profile                             |

### Kafka listeners

- INTERNAL://:9092 advertises kafka:9092 to other containers.
- EXTERNAL://:29092 advertises localhost:29092 to services run directly on the host.
- CONTROLLER://:9093 is the KRaft controller quorum listener.
- Replication factors are 1 because this is a single-node development broker.

### Dockerfile stages

The build stage uses Maven 3.9.9 plus Temurin Java 21 and packages the selected SERVICE plus dependencies. The runtime stage uses a smaller Java 21 JRE, copies only the executable JAR, runs as non-root UID 10001, caps heap ergonomics at 75% of container RAM, and exits on OOM.

#### Efficiency trade-off

Each service image build copies the entire multi-module repository and recompiles dependencies. This is simple and reproducible, though cache-aware per-module Dockerfiles could reduce CI time.

## 23. How to run and verify locally

### Option A: infrastructure in containers, Java services in terminals

```
docker compose up -d postgres redis kafka
mvn clean package -DskipTests

# Run each in a separate terminal:
mvn -pl event-service spring-boot:run
mvn -pl reservation-service spring-boot:run
mvn -pl payment-service spring-boot:run
mvn -pl booking-service spring-boot:run
mvn -pl gateway-service spring-boot:run
```

### Option B: everything in Compose

```
docker compose up -d --build
docker compose ps
docker compose logs -f --tail=200
```

### Health checks

```
for port in 8080 8081 8082 8083 8084; do
  curl --fail "http://localhost:${port}/actuator/health"
done
```

### Safe static verification without containers

```
JAVA_HOME=/opt/homebrew/opt/openjdk@21 mvn -B -DskipTests verify
docker compose config --quiet
git diff --check
```

### Clean shutdown

```
docker compose down          # retain PostgreSQL volume
docker compose down -v       # delete local database volume and reset seed state
```

#### Your Mac

Rancher Desktop or another compatible runtime can provide the Docker engine. The Homebrew docker-compose plugin supplies Compose. Docker Desktop is not required.

## 24. Manual end-to-end walkthrough

### 1. Read seeded data

```
curl http://localhost:8080/api/events
curl http://localhost:8080/api/events/10000000-0000-0000-0000-000000000001/seats
```

### 2. Hold seat A-1

```
curl -i -X POST http://localhost:8080/api/reservations/holds \
-H 'Content-Type: application/json' \
-H 'Idempotency-Key: learn-hold-1' \
-d '{"eventId":"10000000-0000-0000-0000-000000000001", "seatId":"20000000-0000-0000-0000-000000000001", "customer":{}}
```

Copy holdId from the JSON response.

### 3. Book the held seat

```
curl -i -X POST http://localhost:8080/api/bookings \
-H 'Content-Type: application/json' \
-H 'Idempotency-Key: learn-booking-1' \
-d '{"holdId":"<hold-id>", "customerId":"30000000-0000-0000-0000-000000000001", "amountMinor":499900,"currency":{}}
```

### 4. Observe asynchronous projection

Repeat the seat-map GET. After Kafka processing, A-1 should become BOOKED. Also query GET /api/bookings/{bookingId} and GET /api/reservations/holds/{holdId}.

### 5. Repeat requests

Repeat the same hold and booking commands with identical keys/payloads. You should receive the original IDs. Change a payload but keep its key; expect IDEMPOTENCY\_KEY\_REUSED.

## 25. GitHub Actions: what runs in the cloud

### Build workflow on every push to main and every pull request

| Job                 | Runner action                                                      | What it proves                                                    |
|---------------------|--------------------------------------------------------------------|-------------------------------------------------------------------|
| compile             | Checkout, install/cache Temurin 21, mvn package with tests skipped | All modules and generated protobuf compile/package on clean Linux |
| container matrix x5 | Buildx builds each SERVICE image without push                      | Dockerfile produces all five runnable images                      |

### Where the technology comes from

GitHub provides a temporary Ubuntu virtual machine with common tools and Docker. setup-java installs Java 21. Maven downloads dependencies from repositories and caches them. Docker Compose pulls PostgreSQL, Redis, Kafka, and k6 images. Nothing continues running after the job; the runner is discarded.

### Why six checks appear

There is one compile job plus five matrix expansions, one for each service image. A red 1/6 means one check failed, not necessarily that six separate workflows exist.

### Dependabot branches

Dependabot reads the weekly configuration and opens isolated pull requests for Maven and GitHub Action updates. Each branch triggers the same build checks. A failed upgrade PR does not break main until you merge it.

### Current evidence

At commit fd370bc, the hosted build completed successfully. The repository remote is <https://github.com/Agrim2411/SeatSync>.



## 26. Contention benchmark: what it proves

### How to launch

GitHub Actions -> cloud contention benchmark -> Run workflow -> choose 100, 500, 1000, or 2000. It is workflow\_dispatch, so ordinary pushes do not automatically spend benchmark minutes.

### Hosted execution path

#### benchmark.yml

| # | From         | To                | Action / code boundary                                       |
|---|--------------|-------------------|--------------------------------------------------------------|
| 1 | GitHub       | Checkout          | Create a clean copy at the selected commit                   |
| 2 | Compose      | Reservation stack | Build/start reservation plus PostgreSQL, Redis, Kafka        |
| 3 | Shell        | Health endpoint   | Poll up to 60 times, five seconds apart                      |
| 4 | k6 container | Reservation :8082 | One iteration per virtual user against one seat              |
| 5 | k6           | Thresholds        | Require one 201 winner, only 201/409 outcomes, p95 under 2 s |
| 6 | Workflow     | JSON              | Verify file is non-empty and upload artifact for 30 days     |
| 7 | Workflow     | Compose           | Always stop stack and delete volumes                         |

### k6 script details

- Each virtual user gets a unique deterministic customer UUID.
- Every request gets a unique idempotency key, so this measures contention rather than retries.
- 201 increments seat\_hold\_won; 409 increments unavailable; anything else is unexpected.
- The target is reservation port 8082, bypassing gateway rate limiting intentionally.

### What success does and does not mean

| It proves for that run                            | It does not prove                                 |
|---------------------------------------------------|---------------------------------------------------|
| Exactly one hold won the selected hot-seat burst  | No bug can exist in every workload                |
| All other responses were expected conflicts       | Production throughput or capacity                 |
| Configured p95 threshold passed                   | A universal latency SLA                           |
| The tested commit/image worked on GitHub's runner | Multi-region or broker/database high availability |

#### Resume rule

Quote concurrency and latency only from a retained JSON artifact together with commit SHA, date, environment, and workflow URL. Never present the threshold itself as an achieved result.

## 27. Observability and operations

### Implemented hooks

- Every service includes Spring Boot Actuator and the Prometheus registry.
- Health, info, and prometheus endpoints are exposed; gateway also exposes gateway diagnostics.
- Prometheus scrapes all five services every 10 seconds when the observability profile is enabled.
- Gateway returns X-Correlation-ID, though event envelopes do not yet persist it.
- Workers log deferred payment/refund work and failed hold release.

### Signals to monitor

| Signal                              | Why                                                |
|-------------------------------------|----------------------------------------------------|
| Hold success vs 409 contention rate | Healthy demand should not look like server failure |
| Reservation p50/p95/p99             | Hot rows, Redis delay, database saturation         |
| Expired ACTIVE backlog              | Expiry worker is stuck                             |
| Oldest unpublished outbox           | Kafka/publisher delivery is stuck                  |
| Consumer lag and DLT count          | Seat map is stale or poison messages exist         |
| PENDING / PAYMENT_UNKNOWN age       | Checkout reconciliation is stuck                   |
| REFUND_PENDING age                  | Financial compensation risk                        |
| DB pool utilization                 | Connection starvation                              |

### Scaling statement

Gateway and event reads are stateless enough to scale horizontally. Reservation instances share Redis and PostgreSQL, so atomic scripts and constraints still arbitrate. Outbox SKIP LOCKED supports multiple publishers. The single PostgreSQL primary is the deliberate write scaling boundary; eventId-based partitioning is a future option only when telemetry justifies it.

## 28. Failure matrix you should be able to explain

| Failure                             | Current behavior                                   | Correctness impact                               |
|-------------------------------------|----------------------------------------------------|--------------------------------------------------|
| Gateway Redis down                  | Rate limiter may reject/fail requests              | No seat state is changed                         |
| Reservation Redis down              | New holds fail closed                              | No oversell; availability may be reduced         |
| Reservation PostgreSQL down         | Mutation fails                                     | No cache-only booking allowed                    |
| Kafka down                          | Domain transaction commits with unpublished outbox | Seat truth correct; read model stale             |
| Outbox publisher crashes after send | Event can be sent again                            | Inbox prevents duplicate projection effect       |
| Poison Kafka message                | Retry three times, then DLT                        | Projection misses/stales until operator repairs  |
| Payment call times out              | Booking -> PAYMENT_UNKNOWN                         | Worker reconciles provider record                |
| Booking crashes after PENDING       | Stale scan queries payment after grace             | Continue or fail/release                         |
| Payment authorized, hold expired    | Confirmation fails; refund is attempted            | No paid seat confirmation                        |
| Refund call fails                   | Keep REFUND_PENDING                                | Worker retries idempotently                      |
| Hold release gRPC fails             | Log; allow TTL expiry                              | Seat stays unavailable temporarily, not oversold |
| Event read model stale              | Client may see old availability                    | Reservation authority still rejects invalid hold |

### Consistency vs availability choice

SeatSync generally fails closed for mutations. It prefers rejecting or delaying a purchase over risking duplicate ownership. Read availability is more available and may be stale because it does not decide ownership.

## 29. Security boundaries

| Area                  | Implemented                               | Before public deployment                                |
|-----------------------|-------------------------------------------|---------------------------------------------------------|
| Client authentication | JWT resource server in prod profile       | Configure trusted issuer, audience policy, scopes/roles |
| Development           | Permit all, CSRF disabled                 | Never expose dev profile publicly                       |
| Service network       | Plain HTTP/gRPC; ports exposed locally    | Private network plus TLS/mTLS or workload identity      |
| Secrets               | Known local Compose DB password           | External secret manager and rotation                    |
| Payment data          | Only simulator directives like pm_success | Use provider tokens; never accept PAN/CVV here          |
| Actuator              | Health/info/prometheus exposed            | Restrict management endpoints to operations network     |
| Transport             | Plaintext local                           | TLS at ingress and for shared dependencies              |

### Threat-oriented reasoning

- A forged customerId is not prevented in the dev profile; production authentication must bind identity to commands.
- Idempotency keys prevent duplicate effects but are not credentials.
- The Redis ownership token protects cleanup correctness, not user authentication.
- Rate limiting reduces abuse/load but does not replace authorization.
- Public internal service ports would bypass gateway policy: they must be private in real deployment.

#### Portfolio answer

Security controls are intentionally scoped. I implemented the gateway integration point and documented trust boundaries, but I do not claim the default Compose profile is internet-safe.

## 30. Known limitations and defensible improvements

Owning a project means naming its trade-offs. These are not reasons to hide the project; they are opportunities to show engineering judgment.

| Current limit                        | Why acceptable here                                      | Production improvement                                                                    |
|--------------------------------------|----------------------------------------------------------|-------------------------------------------------------------------------------------------|
| No automated test sources            | Earlier scope emphasized compilation/manual verification | Add repository integration tests with Testcontainers and focused state-machine unit tests |
| Payment is a deterministic simulator | Demonstrates orchestration without real credentials      | Provider adapter, webhooks, ledger, secure tokens, reconciliation jobs                    |
| Broad gRPC failure triggers refund   | Simple compensation demonstration                        | CONFIRMATION_UNKNOWN state; query/retry hold before refund                                |
| Kafka key is holdId                  | Read model is non-authoritative                          | Key by seatId and reject stale seat versions                                              |
| Worker rows are not claimed          | Idempotent effects tolerate duplicates                   | SKIP LOCKED claims, leases, or optimistic @Version                                        |
| Correlation not in event envelope    | HTTP tracing still works at gateway                      | Persist correlationId in outbox and propagate to Kafka logs                               |
| One PostgreSQL container/primary     | Simple local portfolio topology                          | Managed HA databases; partition reservation ownership by event                            |
| Single-node Kafka                    | Local/CI reproducibility                                 | Multi-broker replication, ACLs, TLS, topic provisioning                                   |
| Internal ports unauthenticated       | Local debugging only                                     | Network policies plus service identity                                                    |
| No WebSockets or Helm                | Removed to keep five-service code ownable                | Add only when a real product requirement justifies them                                   |

### How to answer 'Why did you not add X?'

I optimized for a coherent correctness story that I could verify and operate. Every extra component creates failure modes and ownership cost. I kept the minimal mechanisms needed to demonstrate concurrency, durable workflows, asynchronous projection, compensation, and operational evidence.

## 31. Debugging playbook

| Symptom                    | First checks                                                     | Likely boundary              |
|----------------------------|------------------------------------------------------------------|------------------------------|
| Gateway returns 429        | Redis health; rate-limit key; route config                       | Gateway admission            |
| Hold returns 409           | Expected contention vs ApiError code; inventory state; Redis key | Reservation decision         |
| Seat map still AVAILABLE   | Kafka broker, outbox published_at, consumer lag/DLT, event_inbox | Async projection             |
| Booking PAYMENT_UNKNOWN    | Payment logs/row by bookingId; wait worker; grace setting        | Uncertain authorize result   |
| Booking REFUND_PENDING     | Payment availability and payment state; worker logs              | Compensation                 |
| Hold never expires         | Scheduler enabled; expires_at; DB lock; processor logs           | Expiry worker                |
| gRPC unavailable           | Reservation 9082 mapping; host env; reservation logs             | Internal command channel     |
| Service startup fails      | DB exists; Flyway output; env URL; schema validate error         | Infrastructure/config        |
| GitHub build red           | Open failed matrix job and first failing command                 | Compile or one service image |
| Benchmark artifact missing | Verify k6 step and JSON verification; failure logs               | Workflow evidence path       |

### Useful commands

```

docker compose ps
docker compose logs --tail=300 reservation-service booking-service payment-service
docker compose logs --tail=300 kafka event-service
curl http://localhost:8082/actuator/health
curl http://localhost:8083/actuator/prometheus
git status --short --branch
mvn -B -DskipTests verify

```

### Database questions to ask

- What is the authoritative row right now?
- Did the domain row and outbox row commit together?
- Is the booking waiting in PENDING, PAYMENT\_UNKNOWN, or REFUND\_PENDING?
- Did the payment row commit even though the caller timed out?
- Is the event inbox present, proving the projector already handled the message?

## 32. Interview questions and strong answers

### Q1. How do you prevent double booking?

Redis atomically admits one contender quickly, but PostgreSQL is authoritative. A conditional update changes AVAILABLE to HELD only if the row is still available, and a partial unique index prevents two ACTIVE holds. Confirmation changes HELD to BOOKED under a locked hold transaction. Therefore Redis loss may increase database traffic, but it cannot create two winners.

### Q2. Why not use only Redis?

Redis is excellent for atomic low-latency admission and TTL, but cache loss/failover/persistence choices make it a poor final authority for a paid seat. PostgreSQL supplies durable transactions and constraints. I accept a database write boundary to preserve correctness.

### Q3. Why both Redis and PostgreSQL?

Redis rejects most hot-key losers before they contend on the database row. PostgreSQL remains the correctness backstop. This separates performance optimization from business truth.

### Q4. What is the transactional outbox solving?

It removes the database/Kafka dual-write gap. The domain transition and event intent commit together. Publishing can retry and may duplicate, so the consumer uses an inbox.

### Q5. Is Kafka exactly once?

No. The system explicitly uses at-least-once delivery. Exactly-once business effects come from idempotency and uniqueness at consumers, not from claiming the transport never duplicates.

### Q6. Why gRPC?

Booking-to-reservation is an internal typed command with a small protobuf schema and bounded deadline. Public APIs remain REST. The choice demonstrates contract generation without forcing gRPC everywhere.

## 33. Interview questions: failures and scaling

### Q7. What happens when payment times out?

Timeout is an unknown outcome, not failure. Booking stores `PAYMENT_UNKNOWN`. A worker queries payment by `bookingId`. If authorized, it confirms; if declined, it releases; if absent beyond the grace period, it fails and releases.

### Q8. What happens if booking crashes mid-request?

`PENDING` is committed before payment. On restart, the worker scans stale `PENDING` rows after ten seconds and reconciles against payment state. This closes the crash window without racing a healthy checkout.

### Q9. What if payment succeeds after the hold expires?

Reservation rejects confirmation because the hold is no longer `ACTIVE/unexpired`. Booking enters `REFUND_PENDING` and invokes an idempotent refund, with scheduled retries until `REFUNDED`.

### Q10. What happens if Kafka is down?

Reservation commits with an unpublished outbox row. The seat authority remains correct, while the event read model becomes stale. Publishing resumes when Kafka recovers.

### Q11. How does it scale?

Stateless gateway and read services scale horizontally. Reservation instances coordinate through Redis and PostgreSQL; publishers share outbox work with `SKIP LOCKED`. The single write primary is the boundary. At larger scale I would partition reservation ownership by `eventId` after telemetry proves the need.

### Q12. What would you improve first?

First add integration tests around concurrency, expiration, and saga recovery. Then model confirmation-unknown separately so a lost gRPC response is queried before refunding. For the read model, key/order events by seat version.



## 34. Interview questions: design judgment

### Q13. Why microservices for a personal project?

The goal was to learn distributed failure boundaries. I kept only five business-owned services so each split demonstrates something concrete: edge policy, read projection, reservation authority, orchestration, and payments. I would choose a modular monolith for a small product team until independent scaling or ownership justified this split.

### Q14. Is the event service the source of truth?

No. It is a query projection. The reservation database decides ownership. This lets reads be asynchronous without risking overselling.

### Q15. Why fail closed when Redis is down?

For scarce paid inventory, availability is less important than preventing incorrect ownership. A failed gate returns an error rather than bypassing the intended contention path. PostgreSQL correctness would still hold if we deliberately designed a DB-only fallback, but current code does not.

### Q16. What does the benchmark demonstrate?

For a documented commit and runner, N distinct customers contended for one fresh seat and exactly one received 201; the rest received expected 409s, and the latency threshold passed. It is a correctness/load scenario, not a general capacity claim.

### Q17. What is eventual consistency here?

Reservation state commits first. Kafka/outbox delivery updates the event seat map later. During lag, reads can be stale. Commands still validate against reservation truth.

### Q18. Where are your transaction boundaries?

Each service has local PostgreSQL transactions only. Create hold atomically updates inventory, hold, idempotency, and outbox. There is no distributed transaction across booking, payment, and reservation; that workflow uses durable states, idempotency, and compensation.

## 35. Whiteboard narratives

### Five-minute no-double-booking narrative

- Start with invariant: one active hold and one confirmed booking per seat.
- Draw Client -> Gateway -> Reservation.
- Draw Redis as an admission optimization, then PostgreSQL as authority.
- Show Lua SET-if-absent and SQL AVAILABLE -> HELD affected-row check.
- Add partial unique index and transaction boundary around hold/idempotency/outbox.
- Show expiration and confirmation competing on a pessimistically locked hold.
- Finish with Kafka projection being stale but non-authoritative.

### Five-minute checkout narrative

- Create durable PENDING booking with unique hold and idempotency key.
- Authorize payment using booking-derived key.
- On authorization, confirm hold over typed gRPC; then mark booking CONFIRMED.
- Decline releases hold. Timeout becomes PAYMENT\_UNKNOWN. Crash leaves stale PENDING.
- Worker queries provider state and retries idempotent effects.
- Confirmation failure creates REFUND\_PENDING and idempotent compensation.

### Thirty-second architecture summary

Five Spring Boot services separate edge policy, read projection, seat authority, booking orchestration, and payment simulation. PostgreSQL enforces correctness, Redis absorbs contention, Kafka updates an idempotent read model through an outbox, and gRPC carries internal reservation commands. Durable saga states recover timeouts and crashes.

## 36. File-by-file reading map: platform, gateway, event

| File                                              | Read it for                                                      |
|---------------------------------------------------|------------------------------------------------------------------|
| pom.xml                                           | Module list, Java 21, dependency/plugin versions                 |
| platform-contracts/.../ApiError.java              | Shared HTTP error shape                                          |
| platform-contracts/.../EventEnvelope.java         | Intended generic event metadata; currently not used by publisher |
| platform-contracts/.../reservation_commands.proto | Generated gRPC contract                                          |
| gateway/.../GatewayApplication.java               | Boot entry point                                                 |
| gateway/.../GatewayConfiguration.java             | Correlation and rate-limit identity                              |
| gateway/.../DevSecurity.java                      | Unauthenticated local profile                                    |
| gateway/.../ProductionSecurity.java               | JWT resource-server profile                                      |
| gateway/.../application.yml                       | Routes, limits, Redis, issuer config                             |
| event/.../Event.java                              | Event catalog entity                                             |
| event/.../Seat.java                               | Seat-map projection entity                                       |
| event/.../EventController.java                    | Read API entry points                                            |
| event/.../SeatRepository.java                     | Ordered query and availability update                            |
| event/.../ReservationEventProjector.java          | Inbox and Kafka projection                                       |
| event/.../KafkaFailureConfiguration.java          | Retries and DLT                                                  |
| event/.../V1__event_catalog.sql                   | Projection schema and sample rows                                |

## 37. File-by-file reading map: reservation

| File                                     | Read it for                                                 |
|------------------------------------------|-------------------------------------------------------------|
| ReservationServiceApplication.java       | Scheduling-enabled Boot entry point                         |
| ReservationController.java               | Public hold HTTP endpoints                                  |
| ReservationDtos.java                     | Validated request and response records                      |
| ReservationService.java                  | Core hold, confirm, release, idempotency, transaction logic |
| RedisSeatGate.java                       | Atomic acquire and compare-delete Lua scripts               |
| SeatInventoryRepository.java             | Conditional AVAILABLE/HELD/BOOKED SQL                       |
| SeatHoldRepository.java                  | Pessimistic row lock and expiry scan query                  |
| SeatInventory.java / SeatState.java      | Authoritative inventory mapping                             |
| SeatHold.java / HoldStatus.java          | Hold aggregate and transitions                              |
| IdempotencyRecord.java                   | Key, request hash, hold link                                |
| OutboxEvent.java / OutboxRepository.java | Durable event intent and SKIP LOCKED batch                  |
| OutboxPublisher.java                     | Kafka envelope/send/published marker                        |
| HoldExpiryProcessor.java                 | Scheduler plus per-hold locked expiration                   |
| ReservationGrpcEndpoint.java             | Generated service implementation and server bean            |
| ReservationErrorHandler.java             | Domain/validation error to HTTP mapping                     |
| application.yml                          | TTL, scan, gRPC, Kafka, Redis, database settings            |
| V1__reservation_schema.sql               | Constraints, indexes, seed inventory                        |

## 38. File-by-file reading map: booking and payment

| File                             | Read it for                                             |
|----------------------------------|---------------------------------------------------------|
| BookingController.java           | Create/get HTTP endpoints and response status choice    |
| BookingDtos.java                 | Checkout/payment wire records and validation            |
| Booking.java                     | Saga state machine fields and mutations                 |
| BookingStore.java                | Local transactions, idempotency, state persistence      |
| BookingRepository.java           | Replay and worker scan queries                          |
| BookingOrchestrator.java         | Payment REST, gRPC confirm/release, refund compensation |
| PaymentReconciliationWorker.java | PENDING/UNKNOWN/refund scheduled recovery               |
| GrpcClientConfiguration.java     | ManagedChannel and HTTP timeout setup                   |
| booking application.yml          | Service addresses, worker interval, grace period        |
| V1__booking_schema.sql           | Booking uniqueness and allowed states                   |
| PaymentController.java           | Internal authorize/find/refund API                      |
| PaymentService.java              | Idempotency, latency modes, payment decisions           |
| Payment.java                     | AUTHORIZED/DECLINED/REFUNDED state                      |
| PaymentRepository.java           | Lookup by key and booking                               |
| V1__payment_schema.sql           | One payment per booking constraints                     |

## 39. File-by-file reading map: infrastructure and evidence

| File                            | Read it for                                                      |
|---------------------------------|------------------------------------------------------------------|
| compose.yml                     | All runtime containers, DNS names, ports, dependencies, profiles |
| Dockerfile                      | Multi-stage Java 21 image build and non-root runtime             |
| infra/postgres/init.sql         | Four logical databases                                           |
| infra/prometheus/prometheus.yml | Five actuator scrape targets                                     |
| .github/workflows/build.yml     | Java package and five-image matrix                               |
| .github/workflows/benchmark.yml | Manual hosted contention evidence                                |
| .github/dependabot.yml          | Weekly dependency/action pull requests                           |
| load-tests/contention.js        | One-seat, one-attempt-per-VU invariant test                      |
| load-tests/README.md            | Reproduction and evidence rules                                  |
| docs/INVARIANTS.md              | Non-negotiable correctness statements                            |
| docs/OPERATIONS.md              | Signals, failures, alerts, scaling                               |
| docs/MANUAL_VERIFICATION.md     | Human verification scenarios                                     |
| docs/adr/0001...                | Why PostgreSQL is authoritative                                  |
| docs/adr/0002...                | Why delivery is at least once                                    |
| SECURITY.md                     | Personal-project deployment boundaries                           |

## 40. Glossary

| Term                 | Meaning in this project                                                   |
|----------------------|---------------------------------------------------------------------------|
| Invariant            | A property that must remain true under all supported behavior             |
| Source of truth      | The state used to make the final business decision                        |
| Admission gate       | Fast preliminary filter that does not own final truth                     |
| Conditional update   | SQL update that succeeds only if current state matches                    |
| Idempotent           | Safe to repeat without creating another business effect                   |
| Hot key / hot row    | One seat receiving many simultaneous requests                             |
| Pessimistic lock     | Database lock taken before deciding a state transition                    |
| Transactional outbox | Event-intent row committed with domain state                              |
| Inbox                | Consumer table recording event IDs already applied                        |
| At least once        | A message is not lost but may be delivered more than once                 |
| Eventual consistency | A read model converges after asynchronous delivery                        |
| Compensation         | A new action, such as refund, that semantically reverses a completed step |
| Saga                 | Distributed workflow using local transactions and durable states          |
| Reconciliation       | Background comparison of local workflow state with provider state         |
| KRaft                | Kafka metadata quorum mode without ZooKeeper                              |
| DLT                  | Dead-letter topic for messages that exhausted retries                     |
| p95                  | 95 percent of measured requests completed at or below this latency        |

## 41. Seven-day ownership plan

| Day | Study                                             | Proof you own it                                              |
|-----|---------------------------------------------------|---------------------------------------------------------------|
| 1   | Sections 1-7; run health and read APIs            | Draw architecture and name every service/database from memory |
| 2   | Sections 8-11; create/replay/contend for holds    | Explain all three no-double-booking layers                    |
| 3   | Sections 12-14; inspect outbox and seat map       | Explain at-least-once plus inbox and expiry race              |
| 4   | Sections 15-20; run success/decline/timeout       | Trace every booking state without notes                       |
| 5   | Sections 21-29; Compose, CI, security, operations | Explain local vs GitHub execution and five failure cases      |
| 6   | Sections 30-34; rehearse limitations and Q&A      | Answer 18 questions aloud in under two minutes each           |
| 7   | Sections 35-39; full code walk                    | Give a 30-minute mock interview with repository open          |

### Final ownership checklist

- I can trace GET seat map, POST hold, and POST booking from gateway to response.
- I can state which database is authoritative for every business object.
- I can explain every hold and booking state transition.
- I can prove why concurrent hold requests have one winner.
- I can explain Redis failure, Kafka failure, payment timeout, process crash, and refund retry.
- I can explain how Compose networking differs from localhost development.
- I can explain what GitHub runners install, build, execute, and discard.
- I can interpret a k6 artifact without turning a one-run result into a capacity claim.
- I can name current limitations and propose focused improvements.
- I can say 'I do not know yet' and then reason from the invariant and source of truth.

#### You are interview ready when

You can explain the system from memory, open any core class and narrate why it exists, predict the durable state after a failure, and distinguish what the code guarantees from what the documentation proposes.

### Repository reference

GitHub: <https://github.com/Agrim2411/SeatSync>

Guide basis: main at fd370bc

Generated: 24 Aug 2026, Asia/Kolkata